

TECNOLOGIA SETÚBAL

ESCOLA SUPERIOR
POLITÉCNICO SETÚBAL

Martim Batel
2024124299
Eric Silva
2024101825
Daniel Avelino
202001676

Projeto de Programação - HONEYPOT

Honeypot com Firewall Dinâmica e Interface Gráfica

UC – Programação Aplicada à Cibersegurança II
Utente – Prof. Filipe Oliveira

25/06/2025

ÍNDICE

1. **INTRODUÇÃO** 3
2. **OBJETIVOS** 4
3. **ARQUITETURA DO SISTEMA** 5
4. **TECNOLOGIAS UTILIZADAS** 7
5. **DESCRIÇÃO DAS FUNCIONALIDADES IMPLEMENTADAS** 8
6. **MANUAL DE INSTALAÇÃO** 13
7. **MANUAL DE UTILIZAÇÃO** 14
8. **CONCLUSÃO** 15
9. **BIBLIOGRAFIA** 16

1. INTRODUÇÃO

O presente relatório detalha o desenvolvimento do projeto realizado no âmbito da unidade curricular de **Programação Aplicada à Cibersegurança II.**

O objetivo central deste trabalho consistiu na implementação de um sistema de honeypot com uma firewall dinâmica, utilizando a linguagem de programação Python.

O sistema foi desenvolvido para ser capaz de capturar tentativas de ataque, analisar atividades consideradas maliciosas e, em simultâneo, utilizar uma firewall que se atualiza automaticamente para bloquear os atacantes detetados.

Neste relatório documentaremos a criação deste projeto, as tecnologias utilizadas, as funcionalidades implementadas e os resultados obtidos.

Este relatório, à semelhança do projeto, foi desenvolvido em grupo pelos alunos Martim Batel, Eric Silva e Daniel Avelino, e a sua submissão foi realizada através da plataforma online GitHub, cumprindo as regras de entrega definidas no enunciado do projeto.

2. OBJETIVOS

A realização deste projeto teve como finalidade cumprir um conjunto de objetivos práticos e de aprendizagem.

Os principais objetivos delineados para o sistema foram:

- **Desenvolver um Honeypot Funcional:** Construir um honeypot em Python capaz de simular um serviço vulnerável (SSH) para capturar tentativas de ataque e registar as interações que acontecessem.

- **Implementar uma Firewall Dinâmica:** Criar uma firewall que se atualiza em tempo real para bloquear automaticamente os endereços IP dos atacantes detetados com base na sua atividade maliciosa.

- **Integrar com Fontes de Threat Intelligence:** Capacitar o sistema para consultar APIs externas (neste caso, AbuseIPDB) para identificar e bloquear de maneira eficaz IPs já conhecidos como perigosos.

- **Construir um Painel de Monitorização:** Desenvolver uma interface gráfica que permita visualizar em tempo real as tentativas de ataque, os IPs bloqueados e controlar o estado do programa.

- **Criar um Sistema de Logs Estruturado:** Estabelecer um mecanismo de registo de atividades que gere logs detalhados e organizados para facilitar a consulta e análise posterior dos eventos de segurança."

3. ARQUITETURA DO SISTEMA

A arquitetura do nosso sistema foi desenhada de forma organizada, dividindo as responsabilidades em componentes lógicos distintos, embora ambos implementados em Python.

O projeto foi dividido nas seguintes camadas:

- **Painel de Monitorização:** Representado pelo ficheiro gui_simples.py, este componente constitui a interface do utilizador. Desenvolvido com a biblioteca Tkinter, é responsável por apresentar visualmente as tentativas de ataque e os bloqueios em tempo real, bem como por permitir ao utilizador iniciar e parar o serviço do honeypot.

- **Núcleo do Honeypot (Backend):** O ficheiro “honeypot.py” serve como o cérebro da aplicação, contendo toda a lógica de negócio.
- Este componente é responsável por:
    - **Simular o Serviço SSH:** Utilizar a biblioteca “Paramiko” para capturar as interações.
    - **Analisar Comportamento:** Detetar ataques de “brute-force” ao contar as tentativas de login falhadas.
    - **Gerir a Firewall Dinâmica:** Manter uma lista de IPs bloqueados que é atualizada dinamicamente sempre que um comportamento malicioso é detetado.
    - **Integrar com Threat Intelligence:** Comunicar com a API do AbuseIPDB para verificar a reputação de um IP antes de processar a sua ligação.

- **Sistema de (Logs):** Para o armazenamento permanente dos dados, o sistema utiliza dois tipos de ficheiros de log:
    - **Tentativas ssh.json:** Guarda de forma estruturada os detalhes de cada tentativa de login (IP, utilizador, password).
    - **honeypot.log:** Regista os eventos mais importantes do sistema (início, paragem, bloqueios, etc.) para fins de auditoria.

O fluxo de funcionamento do sistema inicia-se quando o tráfego da internet chega à firewall dinâmica.

A firewall, que inicialmente permite a passagem das conexões, encaminha o tráfego para o honeypot, que por sua vez regista as interações e toma as decisões de bloqueio.

4. TECNOLOGIAS UTILIZADAS

Para a concretização deste projeto, foi selecionado um conjunto de tecnologias e bibliotecas relacionadas com a linguagem Python, bem como serviços externos, que permitiram a implementação de todas as funcionalidades requeridas.

As principais tecnologias utilizadas foram:

• **Python:** Foi a linguagem de programação central utilizada para o desenvolvimento de toda a lógica do projeto.

• **Tkinter**: É a biblioteca standard da interface gráfica do Python, utilizada para construir o "Painel de Monitorização", permitindo a interação com o utilizador e a visualização de dados em tempo real.

• **Paramiko**: Uma biblioteca Python focada na implementação do protocolo SSHv2. Foi a peça fundamental para a criação do honeypot, permitindo simular um servidor SSH e analisar as tentativas de autenticação.

• Requests**:** Uma biblioteca standard para a realização de pedidos HTTP em Python. Foi utilizada para comunicar com a API externa de Threat Intelligence.

• **AbuseIPDB API:** Um serviço externo de Threat Intelligence que foi integrado no nosso sistema para consultar a reputação de endereços IP com base em relatórios de atividade maliciosa a nível global.

• **Git e GitHub:** O sistema de controlo de versões Git e a plataforma de alojamento de repositórios GitHub foram utilizados para a gestão do código-fonte, trabalho colaborativo e para a submissão final do projeto.

5. DESCRIÇÃO DAS FUNCIONALIDADES IMPLEMENTADAS

Firewall Dinâmica: A firewall dinâmica é um dos pontos centrais do nosso sistema, responsável por bloquear em tempo real os atacantes identificados. A sua implementação foi realizada através da classe FirewallDinamica no ficheiro honeypot.py.

A lógica de funcionamento baseia-se na manutenção de uma lista de IPs bloqueados. Para este fim, foi utilizado um dicionário Python (ips_bloqueados), uma estrutura de dados que permite uma verificação e armazenamento quase instantâneo de um IP.

Quando um comportamento malicioso é detetado, como por exemplo, um ataque de brute-force SSH, o método bloquear_ip é chamado. Este método adiciona o endereço IP do atacante ao dicionário, juntamente com o motivo do bloqueio (ex: "Brute Force SSH" ou "AbuseIPDB") e um registo temporal (timestamp) do momento exato do bloqueio.

Cada nova conexão recebida pelo honeypot é primeiro validada pelo método verificar_bloqueio(ip). Este método consulta o dicionário e caso o IP da conexão for encontrado, a ligação é imediatamente terminada, impedindo o atacante de prosseguir.

Este processo é o que torna a firewall dinâmica: ela adapta-se e aplica novas regras em tempo real, sem que seja necessário parar a aplicação.

Honeypot e simulação do SSH: O principal do nosso sistema é o honeypot, desenvolvido para simular um servidor SSH e assim atrair e analisar atacantes.

Para a implementação deste serviço, recorremos à biblioteca “Paramiko”, uma poderosa ferramenta em Python que implementa o protocolo SSHv2.

A nossa implementação centra-se na classe “HoneypotServer”, que herda da classe “paramiko.ServerInterface”. Esta abordagem permite-nos redefinir os métodos de autenticação para controlar o comportamento do servidor.

O método mais importante é o “check_auth_password”(self, username, password), que tem uma dupla função:

1. Captura de Dados: Todas as credenciais (utilizador e password) fornecidas pelo atacante são capturadas e imediatamente registadas no ficheiro “tentativas_ssh.json”. Este registo estruturado é essencial para a análise posterior das táticas e das credenciais mais comuns utilizadas pelos atacantes.

2. Recusa de Autenticação: Independentemente das credenciais fornecidas, o método retorna sempre um sinal de falha de autenticação ao “Paramiko”. Isto garante que o atacante nunca obtém acesso real ao sistema, enquanto o mantém conectado e a fornecer dados.

Adicionalmente, este método está diretamente ligado à firewall dinâmica.

Ele monitoriza o número de tentativas de login por cada endereço IP. Após três tentativas falhadas, o sistema assume que está a decorrer um ataque de brute-force SSH, e o IP do atacante é automaticamente enviado para a firewall para ser bloqueado.

Integração com Threat Intelligence: Para além de reagir a ataques em tempo real, o nosso sistema foi construído com uma camada de defesa proativa.

Conforme previsto no enunciado, a firewall pode integrar-se com feeds de threat intelligence para prevenir ataques de IPs previamente identificados como maliciosos.

Para esta funcionalidade, escolhemos integrar o nosso sistema com a API do **AbuseIPDB**, uma conhecida plataforma que agrega dados sobre endereços IP envolvidos em atividades abusivas na internet.

A integração foi implementada através da função “verificar_ip_abuseipdb” no ficheiro honeypot.py. Antes de o honeypot processar qualquer interação de um novo endereço IP, esta função é chamada para realizar uma consulta à API do AbuseIPDB.

Com o uso da biblioteca “requests”, o sistema envia um pedido HTTP GET para a API, incluindo o endereço IP a ser verificado e a nossa chave de API para autenticação.

A decisão de bloquear um IP com base na resposta da API é tomada com base em dois critérios:

- Se o **"abuseConfidenceScore"** do IP for igual ou superior a 50 pontos.
- Se o número total de “denuncias” (**"totalReports"**) contra esse IP for superior a 5.

Se qualquer uma destas condições for satisfeita, o IP é considerado malicioso e é imediatamente bloqueado pela nossa firewall dinâmica, antes mesmo de ter a oportunidade de interagir com o serviço SSH simulado.

Trabalhar desta maneira aumenta significativamente a eficácia do sistema, bloqueando atacantes conhecidos de forma eficiente.

Sistema de Logs/Registo de Atividades: Um dos requisitos fundamentais de qualquer sistema de segurança é a sua capacidade de gerar logs detalhados para auditoria e análise.

O nosso projeto implementa dois mecanismos de logging distintos para garantir um registo de atividades eficaz e organizado.

1. Log de Tentativas de Autenticação (tentativas_ssh.json)

Para capturar os dados brutos de cada tentativa de ataque ao serviço SSH, o sistema gera um ficheiro em formato JSON.

Cada vez que um atacante tenta autenticar-se, uma nova linha é adicionada a este ficheiro, contendo um objeto JSON com os seguintes dados:

- O endereço IP do atacante.
- O nome de utilizador (username) inserido.
- A palavra-passe (password) utilizada.
- A data e hora exatas da tentativa.

A escolha do formato JSON foi feita por este ser um formato estruturado, legível tanto por humanos como por máquinas, simplificando assim a importação destes dados para outras ferramentas de análise.

2. Log de Eventos do Sistema (honeypot.log)

Para além do registo de dados brutos, foi implementado um log de eventos do sistema utilizando o módulo logging standard do Python. Este ficheiro funciona como um diário cronológico das ações mais importantes do honeypot.

Cada entrada é formatada com a data, hora, nível do log (ex: INFO) e uma mensagem descritiva. Os eventos registados incluem:

- O arranque e a paragem do serviço do honeypot.
- Conexões permitidas por estarem na whitelist.
- Conexões bloqueadas pela firewall dinâmica.

Esta abordagem de logging permite-nos não só analisar as táticas dos atacantes (através do ficheiro .json), mas também auditar o comportamento e o estado do próprio sistema de honeypot (através do ficheiro .log).

Painel de Monitorização (Interface gráfica): Para permitir a interação e a observação do sistema, foi desenvolvido um painel de monitorização com uma interface gráfica (GUI). Conforme o enunciado, este painel permite visualizar em tempo real as tentativas de ataque e os bloqueios aplicados. A interface foi construída com recurso à biblioteca Tkinter, a toolkit standard para GUIs em Python.

A janela principal do painel, implementada na classe SimpleGUI, é composta por vários elementos interativos:

- **Logs em Tempo Real:** Um campo de texto central que exibe dinamicamente todas as mensagens de atividade do honeypot, desde novas conexões e tentativas de login a bloqueios efetuados pela firewall. Os dados são atualizados instantaneamente sem que seja necessário reiniciar a aplicação.
- **Botões de Controlo:**
    - **"Iniciar Honeypot":** Ativa o servidor SSH e inicia a monitorização.
    - **"Parar":** Interrompe o servidor de forma controlada, terminando a escuta de novas conexões.

Para garantir que a interface gráfica se mantém responsiva e não congela enquanto o servidor está a processar as conexões, a lógica do honeypot é executada numa thread separada. Esta abordagem permite que o painel de monitorização e o sistema funcionem em conjunto de forma eficiente.

6. MANUAL DE INSTALAÇÃO

Este guia destina-se a um utilizador com conhecimentos técnicos que pretenda configurar e executar o projeto num ambiente local a partir do código-fonte disponível no repositório.

1. Pré-requisitos de Software: Antes de iniciar, é necessário garantir que os seguintes softwares estão instalados no sistema:

- **Python 3.x**
- **Git** (para clonar o repositório)

2. Passos de Instalação

a) Clonar o Repositório: Abra um terminal ou linha de comandos e clone o repositório do projeto a partir do GitHub com o seguinte comando:
"git clone https://github.com/GitAttwhore/Projeto-Honeypot---PACII-2025.git"

b) Navegar para a Pasta do Projeto: Aceda à diretoria do projeto que acabou de ser criada: "cd Projeto-Honeypot---PACII-2025" (ou o nome correspondente da pasta)

c) Instalar as Dependências: O projeto depende de algumas bibliotecas externas do Python. Instale-as com o seguinte comando: "pip install paramiko requests"

3. **Execução do Projeto** Após a instalação das dependências, o programa está pronto a ser executado.
   Dentro da pasta do projeto, execute o seguinte comando para iniciar a interface gráfica:" python gui_simples.py"

A aplicação será iniciada, e o honeypot pode ser ativado através da própria interface.

7. MANUAL DE UTILIZAÇÃO

Após iniciar a aplicação, o utilizador terá a opção de interagir com o Painel de Monitorização da seguinte forma:

- **Iniciar o Serviço:** Ao clicar no botão "Iniciar Honeypot", o servidor SSH fica ativo e o sistema começa a monitorizar a porta 2222, registando todas as atividades.

- **Monitorar a Atividade:** A área de texto principal da janela exibirá em tempo real os logs de conexões, tentativas de ataque, bloqueios e outras notificações importantes.

- **Parar o Serviço:** Ao clicar no botão "Parar", o servidor é desativado de forma segura, deixando de aceitar novas conexões.

- **Análise de Logs:** Para uma análise mais detalhada, os ficheiros honeypot.log (eventos do sistema) e tentativas_ssh.json (dados das tentativas de login) podem ser consultados diretamente na pasta do projeto.

8. CONCLUSÃO

Como conlusão, este projeto desenvolvido para a UC de Programação Aplicada à Cibersegurança II foi concluído com sucesso, tendo como resultado um sistema de honeypot funcional.

O projeto desenvolvido em Python deteta e regista tentativas de ataque a um serviço SSH simulado, integrado com APIs de threat intelligence para uma defesa eficaz e utiliza uma firewall dinâmica para bloquear atacantes em tempo real, tal como proposto no enunciado.

Toda esta atividade é apresentada numa interface gráfica, criando assim uma ferramenta de segurança prática e eficaz que responde aos requisitos fundamentais do projeto.

9. BIBLIOGRAFIA

Durante o desenvolvimento deste projeto, foram consultadas diversas fontes de informação incluindo a documentação oficial das bibliotecas utilizadas, material da unidade curricular e fóruns da comunidade de desenvolvimento.

As principais fontes foram:

Material da Disciplina:

- Enunciado do Projeto;
- Apontamentos e material de apoio das aulas teóricas e práticas.

Documentação Oficial:

- **Documentação Oficial do Python:** Principal referência para a sintaxe e os módulos da linguagem.
- **Documentação da Biblioteca Paramiko:** Essencial para a implementação do servidor SSHv2.
- **Documentação da Biblioteca Requests:** Utilizada para consulta sobre a realização de pedidos HTTP.
- **Documentação da API do AbuseIPDB:** Para compreender os endpoints, os parâmetros de pedido e os formatos de resposta do serviço de Threat Intelligence.

Comunidades e Fóruns de Desenvolvimento

- **Stack Overflow:** Plataforma de consulta para a resolução de problemas de programação específicos e erros encontrados durante o desenvolvimento.
- **GitHub:** Utilizado para consulta de exemplos de código e para a gestão do nosso próprio repositório.